

Solución Fuerza Bruta

Idea General

El problema seleccionado corresponde a la optimización de tareas en servidores. Un sistema de cómputo dispone de múltiples servidores encargados de ejecutar tareas con diferentes características.

Cada tarea posee:

- un tiempo de ejecución,
- un nivel de prioridad,
- y posibles dependencias con otras tareas.

El objetivo del sistema es encontrar una distribución eficiente de las tareas entre los servidores, minimizando el tiempo total de ejecución y respetando las restricciones del problema.

Entradas del Problema

Cada tarea contiene:

Elemento	Descripción
Nombre	Identificador de la tarea
Tiempo	Tiempo necesario para ejecutarse
Prioridad	Nivel de importancia
Dependencias	Tareas que deben ejecutarse antes

Restricciones

Dependencias

Una tarea no puede ejecutarse hasta que todas sus dependencias hayan finalizado.

Ejemplo:

T2 depende de T1

Entonces:

T1 debe ejecutarse primero.

Prioridades

Las tareas más importantes deben ejecutarse antes que las de menor prioridad.

- prioridad 1 = más importante.
- prioridad 2 = media.
- prioridad 3 = baja.

Balance de carga

Las tareas deben distribuirse entre servidores evitando sobrecargar uno mientras otros permanecen desocupados.

Salida Esperada

El algoritmo debe mostrar:

- la asignación de tareas a servidores.
- el servidor asignado a cada tarea.
- y el tiempo total mínimo de ejecución.

Conjunto de Datos de Prueba

Caso pequeño

```
tasks = [  
    ("T1", 3, 1, []),  
    ("T2", 5, 2, ["T1"]),  
    ("T3", 2, 1, ["T1"])  
]
```

Caso mediano

```
tareas = [  
    ("T1", 3, 1, []),  
    ("T2", 5, 2, ["T1"]),  
    ("T3", 2, 1, ["T1"]),  
    ("T4", 4, 3, ["T2"]),  
    ("T5", 6, 2, ["T2", "T3"]),  
    ("T6", 3, 1, ["T3"])  
]
```

Caso grande

```
tareas_grande = [  
    ("T1", 3, 1, []),  
    ("T2", 5, 2, ["T1"]),  
    ("T3", 2, 1, ["T1"]),  
    ("T4", 4, 3, ["T2"]),  
    ("T5", 6, 2, ["T2", "T3"]),  
    ("T6", 3, 1, ["T3"]),  
    ("T7", 7, 2, ["T4", "T5"]),  
    ("T8", 2, 3, ["T6"]),  
    ("T9", 5, 1, ["T6"]),  
    ("T10", 4, 2, ["T7", "T8"]),  
    ("T11", 6, 3, ["T9"]),  
    ("T12", 3, 2, ["T10", "T11"])  
]
```

Solución por Fuerza Bruta

Idea General

La solución por fuerza bruta consiste en explorar todas las posibles asignaciones de tareas a servidores para encontrar la mejor distribución posible.

El algoritmo:

- verifica dependencias,
- respeta prioridades,

- y calcula la carga total de cada servidor.

Funcionamiento

La implementación:

1. genera todas las posibles asignaciones.
2. identifica tareas disponibles.
3. verifica dependencias.
4. selecciona tareas por prioridad.
5. calcula cargas.
6. y conserva la mejor solución encontrada.

Análisis de Complejidad

Complejidad Temporal

Si existen:

- n tareas,
- m servidores,

cada tarea puede asignarse a cualquiera de los m servidores.

Por lo tanto:

$m ** n$ combinaciones posibles.

Big O Temporal

$O(m ** n)$

Justificación Paso a Paso

Generación de combinaciones

La función:

```
product(range(num_servidores), repeat=len(tareas))
```

genera todas las posibles asignaciones.

Evaluación de cada combinación

Para cada combinación el algoritmo:

- revisa dependencias.
- obtiene tareas disponibles.
- ordena prioridades.
- y calcula cargas.

Crecimiento exponencial

Cada nueva tarea multiplica el número de combinaciones posibles.

Tareas	Servidores	Combinaciones
3	2	$(2^3 = 8)$
6	2	$(2^6 = 64)$
12	2	$(2^{12} = 4096)$

Complejidad Espacial

El algoritmo almacena:

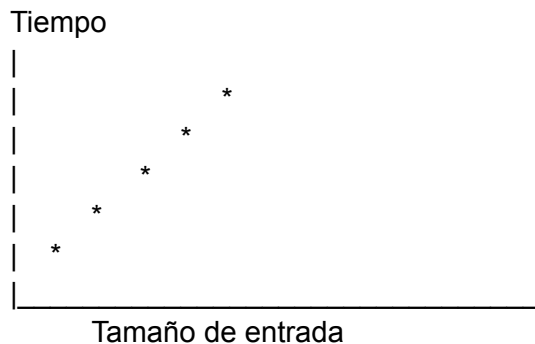
- cargas de servidores.
- tareas ejecutadas.
- asignaciones temporales.

Big O Espacial

$O(n)$

Gráfica Comparativa

La gráfica tiempo vs tamaño de entrada presenta un crecimiento exponencial.



Solución Recursiva

Idea General

La solución recursiva resuelve el problema construyendo las asignaciones paso a paso mediante llamadas recursivas.

En lugar de generar todas las combinaciones directamente, el algoritmo:

- explora decisiones.
- prueba tareas en distintos servidores.
- y construye un árbol de búsqueda.

Caso Base

El caso base ocurre cuando todas las tareas han sido ejecutadas:

```
if len(ejecutadas) == len(tareas):
```

En este punto:

- se calcula el tiempo total.
- y se actualiza la mejor solución encontrada.

Caso Recursivo

El caso recursivo:

1. obtiene tareas disponibles.
2. ordena por prioridad.
3. prueba asignar tareas a cada servidor.

4. realiza llamadas recursivas.
5. y luego deshace cambios para seguir explorando otras soluciones.

Complejidad Temporal

La solución recursiva también explora todas las posibles asignaciones.

Por lo tanto:

$$O(m ** n)$$

Justificación

Aunque la implementación cambia, el algoritmo continúa explorando todas las combinaciones posibles de asignación.

La diferencia principal es que:

- utiliza llamadas recursivas.
- en lugar de generar combinaciones directamente con product.

Complejidad Espacial

La solución recursiva utiliza:

- estructuras auxiliares.
- y además la pila recursiva.

Big O Espacial

$$O(n)$$

Comparación contra Fuerza Bruta

Característica	Fuerza Bruta	Recursiva
Genera combinaciones directamente	Sí	No
Complejidad temporal	$(O(m^n))$	$(O(m^n))$
Complejidad espacial	$(O(n))$	$(O(n))$
Representa árbol de decisiones	No	Sí
Facilidad de implementación	Más simple	Más compleja

Conclusión

La fuerza bruta y la recursividad garantizan encontrar soluciones válidas y óptimas, pero presentan crecimiento exponencial, lo que limita su eficiencia en casos grandes.

La versión recursiva permite modelar mejor el árbol de decisiones y facilita la implementación de futuras optimizaciones como backtracking y poda.